

WORKSHOP PRÁTICO - Semana Acadêmica de Ciência da Computação 2026

Git & GitHub

```
$ git init
```

```
Initialized empty Git repository
```

Sobre mim

- Aluno da Ciência de Computação - Unioeste
- Desenvolvedor Full-Stack @Dilettare



[/in/hugo-gustavo-cordeiro-9ab995209/](https://www.linkedin.com/in/hugo-gustavo-cordeiro-9ab995209/)



[@ugoincc](https://github.com/ugoincc)



hugogustavoc12@gmail.com

Objetivos do Workshop

1

Versionar qualquer projeto

Criar repositório, gravar histórico e navegar por ele

2

Consertar os próprios erros

Desfazer alterações em qualquer um dos três estados do Git

3

Trabalhar em paralelo

Criar branches, mesclar e resolver conflitos sem pânico

4

Colaborar de verdade

Publicar no GitHub e contribuir via Pull Request

Cronograma

DIA 1 — Git na sua máquina

- Por que o Git existe
- Git ≠ GitHub · os 3 estados
- Instalação e configuração
- add · commit · status · log
- .gitignore e como desfazer
- Branches, merge e conflitos

DIA 2 — GitHub e colaboração

- Repositório no GitHub
- Autenticação com token
- push · pull · clone
- README e Markdown
- Fork e Pull Request
- Boas práticas de mercado

ENCONTRO 1 · 2 HORAS

Git na sua máquina

- Tudo hoje funciona sem internet
- Ao final: repositório local com branches mescladas e um conflito resolvido



O problema que o Git resolve

Name

-  Super Cool Report v1.xlsx
-  Super Cool Report v2.xlsx
-  Super Cool Report v3.1.xlsx
-  Super Cool Report v3.xlsx
-  Super Cool Report v4.xlsx
-  Super Cool Report v4a.xlsx
-  Super Cool Report v4b.xlsx
-  Super Cool Report v5.xlsx
-  Super Cool Report vFinal.xlsx
-  Super Cool Report vFinal_1.xlsx
-  Super Cool Report vFinal_2.xlsx
-  Super Cool Report vFinal_Final.xlsx
-  Super Cool Report vFinal_Final-UPDATED.xlsx
-  Super Cool Report vFinal_Final-UPDATED_NEW.xlsx

Qual é o mais recente?

O que mudou entre eles?

Como volto para o de terça?

Git não é GitHub

GIT

- Programa instalado na sua máquina
- Funciona 100% offline
- Criado por Linus Torvalds em 2005
- Guarda e controla o histórico

GITHUB

- Site onde repositórios ficam hospedados
- Exige internet e uma conta
- Empresa — hoje da Microsoft
- Compartilhamento, revisão e social
- Metodologias ágeis - CI e CD

Analogia: Git é o editor de texto. GitHub é o Google Drive onde você guarda o arquivo e convida gente para editar.

Alternativas ao GitHub, com o mesmo Git por baixo: GitLab · Bitbucket · Codeberg

Como o Git enxerga seu projeto

Cada commit é uma fotografia completa



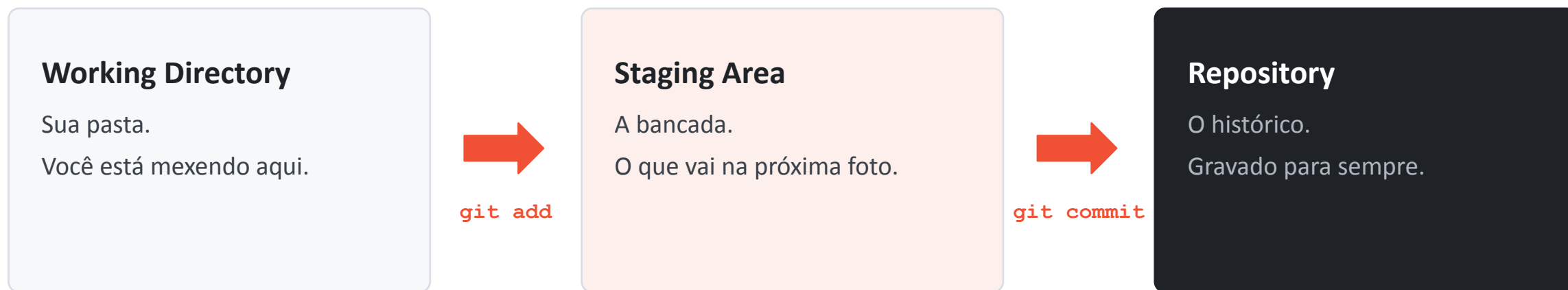
Cada foto guarda: conteúdo completo · autor · data · mensagem · um identificador único (hash)

```
$ git log --oneline  
a3f9c21 Ajusta textos  
7b1e4d0 Corrige validação
```

*Os 7 primeiros
caracteres do hash
já identificam
o commit.*

Os 3 estados — o conceito central do Git

Se você entender este desenho, o resto é vocabulário



Você não fotografa a mesa inteira — escolhe o que entra no quadro. Isso é o staging.

```
$ git status # em qual estado está cada arquivo?
```

Configuração inicial

Uma vez só, por máquina

```
$ git --version

$ git config --global user.name "Nome Sobrenome"
$ git config --global user.email "voce@email.com"
$ git config --global init.defaultBranch main

$ git config --list # confere tudo
```

```
$ git config --global core.editor "nano"
```

Atenção ao e-mail

Use o MESMO e-mail da sua conta do GitHub. É ele que aparece no histórico — e é ele que liga os commits ao seu perfil no Dia 2.

Nunca fique preso no Vim

Defina o editor antes que o Git abra um que você não sabe fechar.

Exercício 0 — Preparando o terreno

- Confirme que o Git está instalado na sua máquina
- Configure seu nome e seu e-mail globais — use o mesmo e-mail da conta do GitHub
- Defina main como nome padrão da branch inicial
- Configure um editor que você saiba fechar
- Liste todas as configurações e confira nome e e-mail

Deu certo se: `git config --list` mostra `user.name` e `user.email` corretos.

O ciclo básico

Estes cinco comandos resolvem 90% do seu dia a dia

```
$ mkdir receitas && cd receitas
$ git init # nasce a pasta .git

$ git status # untracked files: README.md
$ git add README.md
$ git status # to be committed: README.md

$ git commit -m "Adiciona README do projeto"
[main (root-commit) a3f9c21] Adiciona README
```

git init

cria o repositório na pasta atual

git status

mostra o estado de cada arquivo

git add

manda para a área de staging

git commit

grava definitivamente no histórico

git log

mostra o que já foi gravado

git status é o comando mais importante do curso

**Ele não só mostra o estado —
ele diz o que fazer em seguida.**

Quando o aluno travar, a resposta quase sempre está na saída deste comando. Aprender a LER a mensagem vale mais do que decorar o comando seguinte.

```
$ git status
On branch main

Changes not staged for commit:
  (use "git add <file>"...)
  (use "git restore <file>"...)
        modified:   README.md

Untracked files:
  (use "git add <file>"...)
        sobre.md
```

Tudo em laranja acima é o Git literalmente ensinando o próximo comando. Leia sempre.

Escrevendo mensagens que servem para alguma coisa

Você escreve para o você de daqui a seis meses

✗ Não faça

- mudanças
- asdasd
- commit final agora vai
- correções

✓ Faça

- Adiciona validação de e-mail no cadastro
- Corrige cálculo de desconto acima de R\$100
- Remove logs de debug do pagamento

1

Imperativo, no presente

"Adiciona" — nunca "Adicionado" ou "Adicionei"

2

Até ~50 caracteres

cabe na tela, cabe na cabeça

3

Um commit, uma ideia

se a mensagem tem "e", provavelmente são dois commits

Teste mental

"Se aplicado, este commit _____"

Se a frase não fecha, a mensagem está ruim.

Lendo o histórico

Quatro comandos para responder: o que aconteceu aqui?

```
$ git log --oneline # uma linha por commit – o formato do dia a dia
$ git log --oneline --graph --all # desenha as branches

$ git diff # o que mudei e ainda não mandei para o stage
$ git diff --staged # o que já está no stage, pronto para o commit

$ git show a3f9c21 # o conteúdo completo de um commit específico
```

diff é linha a linha

As linhas com + entraram, as com – saíram. É assim que você confere antes de commitar.

Hash resumido

Sempre dá para usar só os 7 primeiros caracteres em vez do identificador inteiro.

Exercício 1 — Seu primeiro repositório

- 1 Crie a pasta meu-portfolio e inicialize o repositório
- 2 Crie README.md com o título "# Meu Portfólio" → commit 1
- 3 Crie sobre.md com duas linhas sobre você → commit 2
- 4 Adicione uma seção "## Projetos" ao README → commit 3
- 5 Veja o histórico com uma linha por commit
- 6 Mostre o conteúdo completo do primeiro commit

Deu certo se: `git log --oneline` lista 3 commits e `git status` diz working tree clean.

.gitignore — o que nunca deve entrar

Higiene do repositório, não detalhe opcional

```
# arquivo: .gitignore
node_modules/
.env
*.log
.DS_Store
/dist
target/
```

.env no repositório é uma senha em post-it — só que o post-it está na internet e tem histórico permanente.

1

Dependências

node_modules, vendor, target — reinstaláveis

2

Segredos

.env, chaves, tokens, senhas

3

Gerados

build, dist, .class, .exe

4

Do sistema

.DS_Store, Thumbs.db

Pegadinha: o .gitignore só vale para arquivos ainda não rastreados. Se já commitou, precisa de `git rm --cached <arquivo>` — e ele continua no histórico.

Desfazendo erros

Escolha o comando pelo estado em que o erro está

Editei e me arrependi

```
git restore arquivo
```

volta ao último commit

Dei add sem querer

```
git restore --staged arquivo
```

sai do stage, edição fica

Errei a mensagem

```
git commit --amend -m "nova"
```

reescreve o último commit

Esqueci um arquivo

```
git add x && git commit --amend --no-edit
```

anexa ao último commit

Quero desfazer o commit

```
git reset --soft HEAD~1
```

desfaz commit, mantém código

Quero apagar tudo

```
git reset --hard HEAD~1
```

destrutivo — sem volta fácil

amend e reset REESCREVEM história. Regra: só reescreva o que ainda não saiu da sua máquina.

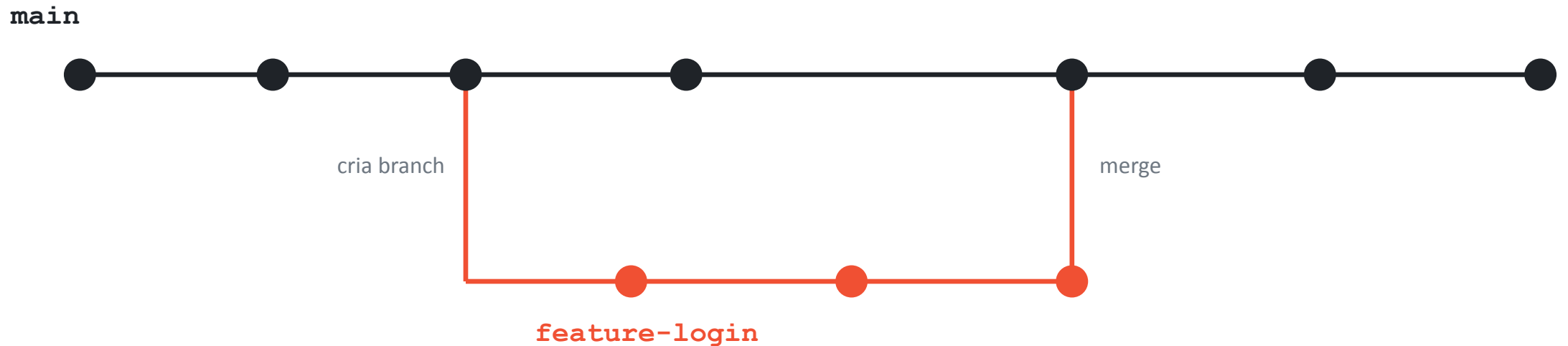
Exercício 2 — Ignorando e desfazendo

- 1 Crie senha.env e uma pasta temp/ com um arquivo dentro
- 2 Crie um .gitignore que ignore *.env e temp/ — confirme que sumiram do git status
- 3 Comite o .gitignore
- 4 Edite o README com qualquer bobagem e descarte a alteração
- 5 Crie rascunho.txt, dê git add e tire do stage sem apagar o arquivo
- 6 Faça um commit com a mensagem "teste" e corrija a mensagem

Deu certo se: senha.env não aparece no status e nenhum commit tem a mensagem "teste".

Branches — linhas do tempo paralelas

Experimente no universo alternativo; se der errado, apague o universo



```
$ git switch -c feature-login # cria e já muda para ela
$ git switch main           # volta para a principal
$ git merge feature-login   # traz as mudanças para cá
$ git branch -d feature-login # limpa o que não usa mais
```

Conflito de merge

Não é erro. É o Git se recusando a chutar por você.

```
$ git merge feature-titulo
CONFLICT (content): Merge conflict in README.md
Automatic merge failed; fix conflicts
```

```
# README.md fica assim:
<<<<<< HEAD
# Portfólio Profissional
=====
# Portfólio de Estudos
>>>>>> feature-titulo
```

<<<<<< HEAD

daqui começa a versão da branch atual

=====

divisor entre as duas versões

>>>>>> outra

aqui termina a versão que está chegando

Escape de emergência

git merge --abort cancela tudo e devolve o estado anterior.

Dois pessoas mexeram na mesma linha. O Git não adivinha qual está certa — quem decide é você.

Resolvendo um conflito em 4 passos

Sempre os mesmos quatro, sempre nesta ordem

1

Abra o arquivo e decida

Pode ficar um lado, o outro, os dois ou algo totalmente novo

2

Apague os três marcadores

<<<<<<< · ===== · >>>>>>> — nenhum pode sobrar

3

git add <arquivo>

É assim que você diz ao Git: resolvi este aqui

4

git commit

A mensagem já vem preenchida — pode aceitar

```
$ git status # se estiver perdido no meio do conflito, ele lista o que falta resolver
```

Exercício 3 — Branch, merge e conflito

- 1 Crie a branch feature-contato, adicione contato.md e comite
- 2 Volte para a main e observe: o arquivo sumiu da pasta
- 3 Mescle feature-contato na main, confirme o retorno do arquivo e apague a branch
- 4 Na main, mude a PRIMEIRA linha do README e comite
- 5 Crie feature-titulo a partir de HEAD~1, mude a MESMA linha e comite
- 6 Volte para a main, tente mesclar → conflito. Resolva juntando as duas versões
- 7 Confirme o desenho com `git log --oneline --graph --all`

Deu certo se: O README não contém nenhum marcador e o grafo mostra as duas linhas se juntando.

Fim do Dia 1

Você já sabe versionar um projeto inteiro sem depender de internet nem de ninguém.

TAREFA DE CASA • 15 minutos

- Versione uma pasta real do seu computador — trabalho, anotações, qualquer coisa
- Faça no mínimo 3 commits com mensagens no formato correto
- Crie uma branch, altere algo e mescle de volta na main
- Traga o resultado de `git log --oneline` no começo do Dia 2

Antes do próximo encontro: crie sua conta no GitHub, com o mesmo e-mail que você configurou hoje.

ENCONTRO 2 • 2 HORAS

GitHub e colaboração

- Hoje seu código sai da sua máquina
- Ao final: repositório publicado e um Pull Request aceito



Revisão

1 Qual comando mostra o estado dos meus arquivos?

2 Qual a diferença entre add e commit?

3 Editei um arquivo e me arrependi. Qual comando?

4 O que significa <<<<<< HEAD dentro de um arquivo?

5 Git e GitHub são a mesma coisa?

Criando um repositório no GitHub

Cinco decisões na hora de criar

1

Nome

sem espaços, minúsculo, com hífen: meu-portfolio

2

Público ou privado

público aparece para todos; privado só para quem você convidar

3

README

não marque agora — vamos enviar o repositório local que já existe

4

.gitignore template

o GitHub gera um pronto para a sua linguagem

5

Licença

define o que os outros podem fazer com o seu código

As quatro abas que importam: Code (arquivos) · Issues (tarefas e bugs) · Pull requests (contribuições em revisão) · Settings

Autenticação: sua senha não funciona mais

Desde 2021 o GitHub exige um token no lugar da senha

1

Settings → Developer settings

2

Personal access tokens → Tokens (classic)

3

Generate new token

4

Marque apenas o escopo repo

5

Expiração: 30 dias

6

COPIE AGORA — aparece uma única vez

```
$ git push
```

```
Username: seu-login
```

```
Password: ghp_xxxxxxxxxx
```

↑ o TOKEN, não a senha

Três avisos

- O token aparece uma vez — perdeu, gera outro
- Nunca comite nem compartilhe o token
- Quando expirar, o push falha: é esperado

Exercício 4 — GitHub e autenticação

- 1 Faça login na sua conta do GitHub
- 2 Crie um repositório PÚBLICO chamado meu-portfolio — sem marcar "Add a README"
- 3 Gere um Personal Access Token com escopo repo e validade de 30 dias
- 4 Guarde o token em local seguro — jamais dentro do projeto

Deu certo se: A página do repositório vazio está aberta com as instruções de push.

Conectando o local ao remoto

Dois caminhos, dependendo de onde o projeto já existe

Já tenho o projeto na máquina

```
$ git remote add origin <url>
$ git remote -v
$ git branch -M main
$ git push -u origin main
# -u apenas na primeira vez
```

O projeto está no GitHub

```
$ git clone <url>
$ cd repo

# pronto: já vem com
# histórico e remoto
```

origin apelido padrão do repositório remoto — convenção, não palavra mágica

push envia os seus commits para o servidor

pull traz os commits do servidor para você

Regra de ouro: pull antes de começar, push depois de terminar.

O ciclo diário — e o erro nº 1 de todo iniciante

```
$ git pull      # SEMPRE primeiro  
# ... trabalha ...  
$ git add .  
$ git commit -m "..."  
$ git push
```

```
$ git push  
! [rejected] main -> main  
error: failed to push  
hint: Updates were rejected  
hint: ... fetch first
```

O que aconteceu

Alguém (ou você mesmo, pelo site) enviou commits que você ainda não tem. O Git se recusa a sobrescrever o trabalho alheio.

Como resolver

```
$ git pull  
$ git push # agora vai
```

A mensagem de erro contém a solução: "fetch first". Ler o erro é a habilidade, não decorar o comando.

Exercício 5 — Publicando e sincronizando

- 1 Conecte o meu-portfolio local ao repositório remoto e confirme com `git remote -v`
- 2 Envie os commits, autenticando com o token
- 3 PELO SITE do GitHub, edite o README, adicione uma linha e comite pelo navegador
- 4 No terminal, crie `skills.md`, comite e tente dar push → vai falhar
- 5 Leia a mensagem de erro, resolva e conclua o push
- 6 Confirme no site que os dois arquivos coexistem

Deu certo se: O repositório remoto tem os dois arquivos e os históricos local e remoto batem.

README — a porta de entrada do seu projeto

Recrutador não lê o seu código. Lê o seu README.

```
# Título      ## Subtítulo
**negrito**  *itálico*  `código`
```

```
- item de lista
1. item numerado
```

```
[texto] (https://url.com)
![imagem] (caminho.png)
```

```
| Coluna | Coluna |
|-----|-----|
```

```
- [x] feito      - [ ] pendente
```

Estrutura mínima

- O que é o projeto, em uma linha
- Funcionalidades
- Tecnologias usadas
- Como rodar
- Autor e contato

Por que importa

É o primeiro — e muitas vezes o único — contato de outra pessoa com o seu trabalho. Vale mais que qualquer linha de código do repositório.

Exercício 6 — README que impressiona

- 1 Título com # e uma linha de descrição do projeto
- 2 Uma seção "## Sobre mim" usando negrito e itálico
- 3 Lista de tecnologias com pelo menos 3 itens
- 4 Bloco de código com o comando para clonar o seu repositório
- 5 Tabela de 2 colunas — por exemplo: Projeto | Descrição
- 6 Link para o seu perfil e uma lista de tarefas com 1 marcada e 1 pendente

Deu certo se: Os sete elementos renderizam corretamente na página do repositório.

Contribuindo em um projeto de outra pessoa

O fluxo padrão do mundo open source



Pull Request é uma conversa, não um botão

- Ele mostra o diff — o revisor vê exatamente o que mudou
- Dá para comentar linha a linha, discutir e pedir ajustes
- Novos commits na branch atualizam o PR automaticamente
- Bom PR explica o PORQUÊ no corpo — o "o quê" já está no diff

Issues — onde o trabalho é combinado

Antes de escrever código, alguém abriu uma issue

1

Reportar bugs

com passos para reproduzir, esperado e obtido

2

Propor funcionalidades

discutir antes de investir tempo codando

3

Organizar com labels

bug · enhancement · good first issue · documentation

4

Fechar pelo commit

escreva `Closes #12` na mensagem e o merge fecha a issue sozinho

Para entrar no open source: procure a label `good first issue` em projetos que você já usa.

Exercício 7 — Pull Request na vida real

- 1 Faça fork do repositório da turma e clone O SEU FORK
- 2 Crie a branch `add-seu-nome`
- 3 Em `/participantes`, crie `seu-nome.md` com nome, uma linha sobre você e o link do portfólio
- 4 Comite: `feat: adiciona perfil de <nome>` e faça push da branch para o seu fork
- 5 Abra o Pull Request para o repositório original, com título e corpo descritivos
- 6 Troque o link do PR com a dupla, revise o diff e deixe um comentário em uma linha específica
- 7 Aprove o PR do colega e, após o merge, sincronize seu fork com o original

Deu certo se: PR aberto, revisado pela dupla e mesclado ainda nesta aula.

Conventional Commits

O padrão que você vai encontrar na maioria das empresas

feat:	adiciona login com Google	nova funcionalidade
fix:	corrige cálculo de frete no Nordeste	correção de bug
docs:	atualiza instruções de instalação	só documentação
refactor:	extrai validação para módulo próprio	muda o código, não o comportamento
test:	adiciona testes do serviço de pagamento	testes
chore:	atualiza dependências do projeto	manutenção geral

Não é regra do Git — é convenção de time. Mas adotar cedo faz seu histórico parecer profissional desde o primeiro projeto.

Sete regras para levar para casa

1 Commit pequeno e frequente vale mais que um commit gigante no fim do dia

2 Uma mudança lógica por commit

3 Nunca comite segredos — .env, chaves, senhas, tokens

4 pull antes de começar, push depois de terminar

5 Branch por funcionalidade — a main fica sempre estável

6 Mensagem no imperativo: o quê no título, o porquê no corpo

7 push --force só na SUA branch, jamais numa branch compartilhada

A sétima é a única que apaga o trabalho dos outros. Por isso ela está em vermelho.

Quando der errado

Os cinco travamentos mais comuns e como sair deles

Terminal travou numa tela cheia de ~

Vim abriu sozinho

Esc → **:wq** → **Enter**

fatal: not a git repository

está fora da pasta ou faltou init

cd para a pasta certa

Please tell me who you are

config sem nome/e-mail

refaça o Exercício 0

! [rejected] ... fetch first

o remoto tem commits novos

git pull e depois git push

Authentication failed

usou a senha em vez do token

gere e use o PAT

\$ git reflog # o salva-vidas: mostra TUDO que aconteceu, inclusive commits "perdidos"

Por onde continuar

Você já tem a base. Daqui em diante é uso diário.

PRÓXIMOS COMANDOS

- `git stash` — guardar trabalho pela metade
- `git rebase` — histórico linear
- `git bisect` — achar o commit que quebrou
- GitHub Actions — automação e CI/CD

ONDE ESTUDAR

- learngitbranching.js.org — visual e interativo
- git-scm.com/book/pt-br — o livro oficial, gratuito
- ohshitgit.com — receitas para sair de enrascadas
- skills.github.com — trilhas práticas do GitHub

OBRIGADO



[/in/hugo-gustavo-cordeiro-9ab995209/](#) - Hugo Gustavo Cordeiro



@ugoincc



hugogustavoc12@gmail.com